INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Tarjetas de lugares con Row, Column, Box y Modifier en Jetpack Compose				
NÚMERO DE PRÁCTICA:	6	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN	10/09/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				**NOTA (0-20)**	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS

Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab06

1. EJERCICIOS RESUELTOS

A continuación se detallan, paso a paso, las instrucciones para construir dos estilos de tarjeta (card) que muestren un lugar del campus, combinando Row, Box, Image y una cadena de Modifier.

Paso 1: Crear un nuevo proyecto.

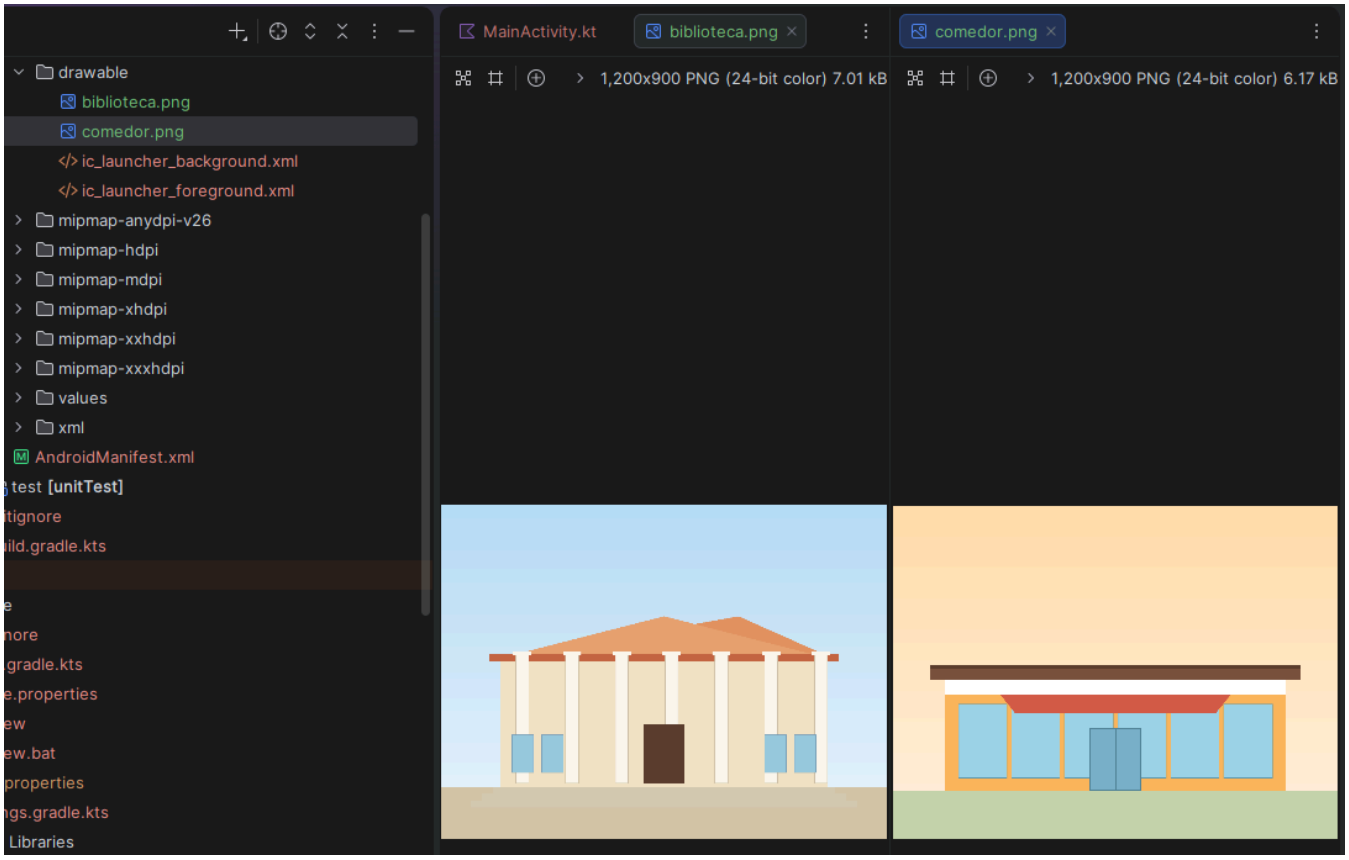
Paso 2: Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

Kotlin

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
```

```
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.painter.Painter
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.tarjetascampus_compose.ui.theme.TarjetasCampus_ComposeTheme
```

Paso 3: Agregar 2 o 3 imágenes de muestra a res/drawable (por ejemplo fotos propias de edificios del campus; si no cuentas con imágenes, puedes usar un Modifier.background(color) como reemplazo temporal del Image)



.

Paso 4: Crear el Composable TarjetaLugar, que recibirá el nombre, la descripción, la imagen y el color de fondo del lugar, y que representará el primer estilo de tarjeta: una franja horizontal con imagen circular a la izquierda. Por ahora dejamos el cuerpo vacío; lo iremos completando en los siguientes pasos:

Paso 5: Dentro de TarjetaLugar, envolver el contenido en un Surface con bordes redondeados que use el color de fondo recibido, y dentro de ese Surface, un Row alineado verticalmente al centro y con un poco de padding:

Paso 6: Dentro del Row, agregar una imagen circular, un espacio y una Column con el nombre (en negrita) y la descripción del lugar, respetando el orden de Modifier resumido en la Figura 2. Con esto, TarjetaLugar queda completo:

Kotlin

```kotlin
// Estilo 1: Tarjeta Horizontal con imagen circular
@Composable
fun TarjetaLugar(
    nombre: String,
    descripcion: String,
    imagen: Painter,
    colorFondo: Color
) {
    Surface(
        shape = RoundedCornerShape(24.dp),
        color = colorFondo
    ) {
        Row(
            verticalAlignment = Alignment.CenterVertically,
            modifier = Modifier.padding(16.dp)
        ) {
            Image(
                painter = imagen,
                contentDescription = nombre,
                contentScale = ContentScale.Crop,
                modifier = Modifier
                    .size(90.dp)
                    .clip(CircleShape)
            )
            Spacer(modifier = Modifier.width(16.dp))
            Column {
                Text(
                    text = nombre,
                    fontWeight = FontWeight.Bold,
                    fontSize = 20.sp
                )
                Text(
                    text = descripcion,
                    fontSize = 14.sp
                )
            }
        }
    }
}
```

Paso 7: Llamar a TarjetaLugar dentro de una Column con espaciado entre elementos, usando al menos dos lugares distintos del campus:

```kotlin
@Composable
fun ListaTarjetasLugar() {
    Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
        TarjetaLugar(
            nombre = "Biblioteca Central",
            descripcion = "Zona de estudio y préstamo de libros",
            imagen = painterResource(id = R.drawable.biblioteca),
            colorFondo = Color(0xFFE3F2FD)
        )
        TarjetaLugar(
            nombre = "Comedor Universitario",
            descripcion = "Servicio de alimentación para estudiantes",
            imagen = painterResource(id = R.drawable.comedor),
            colorFondo = Color(0xFFFFF3E0)
        )
    }
}
```

Paso 8: Crear un segundo Composable TarjetaCompacta, que reciba el nombre del lugar y su imagen, y que represente el estilo apilado: un Box que ocupe todo el ancho disponible, con una altura fija y bordes redondeados, conteniendo una Image de fondo que llene todo el espacio del Box, tal como se resume la superposición de elementos en la Figura 1:

Paso 9: Dentro del mismo Box, agregar un segundo Box semitransparente a modo de scrim, y un Text con el nombre alineado en la esquina inferior izquierda. Con esto, TarjetaCompacta queda completo:

```kotlin
// Estilo 2: Tarjeta Apilada (Box con Scrim e Imagen de fondo)
@Composable
fun TarjetaCompacta(
    nombre: String,
    imagen: Painter
) {
    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(140.dp)
            .clip(RoundedCornerShape(16.dp))
    ) {
        Image(
            painter = imagen,
            contentDescription = nombre,
            contentScale = ContentScale.Crop,
            modifier = Modifier.matchParentSize()
        )
        Box(
            modifier = Modifier
```

```
                .matchParentSize()
                .background(Color.Black.copy(alpha = 0.35f))
        )
        Text(
            text = nombre,
            color = Color.White,
            fontWeight = FontWeight.Bold,
            fontSize = 18.sp,
            modifier = Modifier
                .align(Alignment.BottomStart)
                .padding(12.dp)
        )
    }
}
```

Paso 10: Dentro de MainActivity, en el bloque setContent, mostrar primero la lista de TarjetaLugar (Paso 7) y luego dos TarjetaCompacta apiladas con un Spacer entre ellas:

Kotlin

```
@Composable
fun PantallaTarjetas() {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // Estilo 1: tarjetas con Row (imagen circular + texto)
        ListaTarjetasLugar()

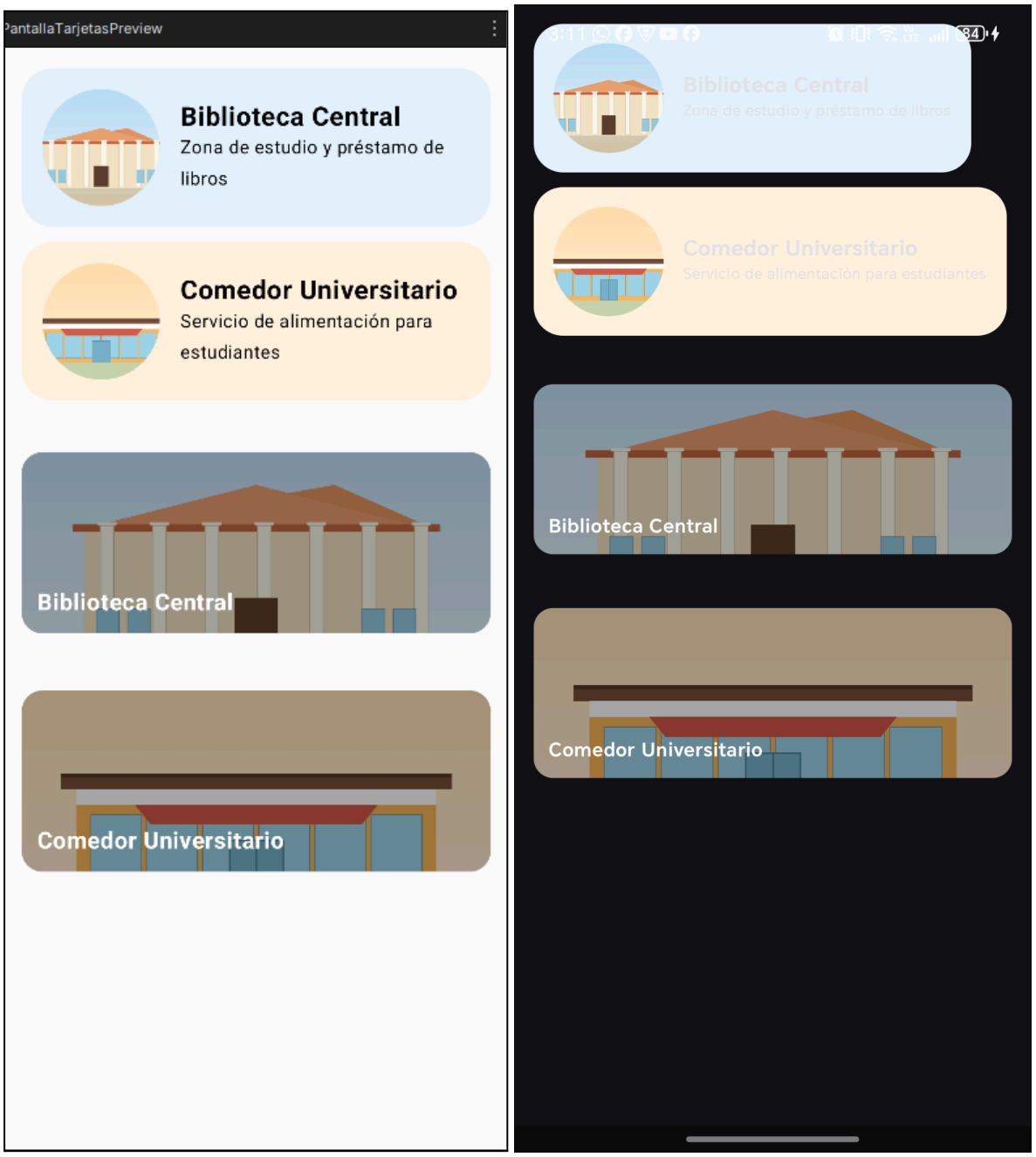
        Spacer(modifier = Modifier.height(8.dp))

        // Estilo 2: tarjetas apiladas con Box (imagen de fondo + scrim + texto)
        TarjetaCompacta(
            nombre = "Biblioteca Central",
            imagen = painterResource(id = R.drawable.biblioteca)
        )
        Spacer(modifier = Modifier.height(12.dp))
        TarjetaCompacta(
            nombre = "Comedor Universitario",
            imagen = painterResource(id = R.drawable.comedor)
        )
    }
}

@Preview(showBackground = true)
@Composable
```

```
fun PantallaTarjetasPreview() {
    TarjetasCampus_ComposeTheme {
        PantallaTarjetas()
    }
}
```

Paso 11: Ejecutar la aplicación en un emulador y comparar visualmente ambos estilos de tarjeta.



2. EJERCICIOS PROPUESTOS|

Tomando como referencia el ejercicio resuelto por el docente, agrega navegación al detalle e integra las tarjetas dentro de la pantalla de edificios del Laboratorio 05:

Se integraron los estilos de tarjeta dentro del proyecto Laboratorio 05 (CampusNavigator_Compose), adaptando la arquitectura existente (Scaffold, NavigationBar, NavHost y SeleccionViewModel).

1. **Crea un Composable DetalleLugarScreen(nombre: String, descripcion: String, imagen: Painter) que muestre la información del lugar a tamaño completo, reutilizando el estilo de TarjetaLugar.**

Se implementó DetalleLugarScreen recibiendo la entidad LugarCampus. Muestra la imagen a pantalla completa con esquinas redondeadas (height(200.dp)), el nombre en tamaño destacado (24.sp), la descripción detallada del edificio y un botón de retorno.

Kotlin

```
@Composable
fun DetalleLugarScreen(
    lugar: LugarCampus,
    onVolver: () -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Surface(
            shape = RoundedCornerShape(24.dp),
            color = lugar.colorFondo,
            modifier = Modifier.fillMaxWidth()
        ) {
            Column(
                modifier = Modifier.padding(16.dp),
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                ImagenOColor(
                    imagenResId = lugar.imagenResId,
                    contentDescription = lugar.nombre,
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(200.dp)
                        .clip(RoundedCornerShape(16.dp))
                )
                Spacer(modifier = Modifier.height(16.dp))
                Text(text = lugar.nombre, fontWeight = FontWeight.Bold, fontSize = 24.sp)
                Spacer(modifier = Modifier.height(8.dp))
                Text(text = lugar.detalleCompleto, fontSize = 16.sp)
            }
        }
        Button(
```

```
            onClick = onVolver,
            modifier = Modifier.fillMaxWidth()
        ) {
            Text("Volver a Edificios")
        }
    }
}
```



2. **Envuelve TarjetaLugar y TarjetaCompacta con Modifier.clickable { ... } y, usando el mismo patrón de navegación del Laboratorio 05 (navController.navigate(...) o una lambda), lleva al usuario a DetalleLugarScreen al pulsar una tarjeta.**

Se agregó el parámetro onClick: () -> Unit a los componentes de tarjeta y se invocó el modificador .clickable { onClick() }. La navegación se gestionó a través de NavHost con la ruta "detalle/{lugarId}", actualizando simultáneamente el estado del edificio seleccionado en SeleccionViewModel.

Kotlin

```
NavHost(
            navController = navController,
            startDestination = Screen.Home.route,
            modifier = Modifier.padding(padding)
        ) {
            composable(Screen.Home.route) {
                HomeScreen(viewModel = viewModel)
            }
            composable(Screen.Edificios.route) {
                EdificiosScreen(
                    onLugarClick = { lugar ->
```

```
                        viewModel.seleccionarEdificio(lugar.nombre)
                        navController.navigate("detalle/${lugar.id}")
                    }
                )
            }
            composable(
                route = "detalle/{lugarId}",
                arguments = listOf(navArgument("lugarId") { type = NavType.StringType })
            ) { backStackEntry ->
                val lugarId = backStackEntry.arguments?.getString("lugarId")
                val lugar = listaLugares.find { it.id == lugarId }
                if (lugar != null) {
                    DetalleLugarScreen(
                        lugar = lugar,
                        onVolver = { navController.popBackStack() }
                    )
                }
            }
```

3. Reemplaza la lista simple de EdificiosScreen (nombre + botón) por una lista de TarjetaLugar o TarjetaCompacta con datos de al menos 4 edificios del campus.

Se creó el modelo LugarCampus conteniendo 4 registros: Biblioteca Central, Comedor Universitario, Pabellón de Sistemas y Polideportivo Campus. En EdificiosScreen se empleó un LazyColumn que renderiza dinámicamente cada tarjeta.

Kotlin

```
val listaLugares = listOf(
    LugarCampus(
        id = "1",
        nombre = "Biblioteca Central",
        descripcion = "Zona de estudio y préstamo de libros.",
        detalleCompleto = "La Biblioteca Central ofrece amplias salas de lectura,
préstamos de libros físicos y digitales, módulos de trabajo grupal y zona de cómputo.",
        colorFondo = Color(0xFFE3F2FD),
        categoria = "Académico",
        imagenResId = R.drawable.biblioteca
    ),
    LugarCampus(
        id = "2",
        nombre = "Comedor Universitario",
        descripcion = "Servicio de alimentación para estudiantes.",
        detalleCompleto = "El Comedor Universitario brinda raciones balanceadas de
alimentos para la comunidad estudiantil con estrictos estándares de calidad.",
        colorFondo = Color(0xFFFFF3E0),
        categoria = "Servicios",
        imagenResId = R.drawable.comedor
```

```
    ),
    LugarCampus(
        id = "3",
        nombre = "Pabellón de Sistemas",
        descripcion = "Laboratorios de software y aulas teóricas.",
        detalleCompleto = "Infraestructura equipada con laboratorios de cómputo de última
generación y aulas para la Escuela Profesional de Ingeniería de Sistemas.",
        colorFondo = Color(0xFFE8F5E9),
        categoria = "Aulas / Labs",
        imagenResId = R.drawable.pabellon_sistemas
    ),
    LugarCampus(
        id = "4",
        nombre = "Polideportivo Campus",
        descripcion = "Espacio para disciplinas deportivas y eventos.",
        detalleCompleto = "Instalaciones deportivas destinadas al entrenamiento de
selecciones universitarias, campeonatos interescuelas y actividades recreativas.",
        colorFondo = Color(0xFFF3E5F5),
        categoria = "Deportes",
        imagenResId = R.drawable.polideportivo
    )
)
```



4. **Diseña un tercer estilo de tarjeta propio (por ejemplo con una etiqueta o ícono adicional) y documenta en el informe qué combinación de Row/Column/Box y Modifier utilizaste.**

Combinación de Layouts:

- Box contenedor exterior: Sirve como marco base para definir el redondeo de bordes (clip(RoundedCornerShape(16.dp))), el color de fondo neutro (Color(0xFFF0F4F8)) y permitir la superposición de capas en el eje Z.
- Column principal: Organiza verticalmente la imagen superior (height(120.dp)) y un bloque interno de textos (Column) con padding de 12.dp para el nombre y la descripción.
- Surface superpuesto (Etiqueta/Badge): Se ubica flotando sobre la esquina superior izquierda de la imagen gracias al uso de Modifier.align(Alignment.TopStart) dentro del Box principal.

Modifiers utilizados:

- Modifier.fillMaxWidth() y Modifier.clip(RoundedCornerShape(16.dp)) para un encuadre limpio.
- Modifier.clickable { onClick() } para la respuesta táctil.
- Modifier.align(Alignment.TopStart) para el posicionamiento absoluto del badge sobre la imagen sin alterar el flujo del texto inferior.



CUESTIONARIO:

1. **¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (dos estilos de tarjeta de lugares: una con Row/Column mostrando imagen y texto, y otra apilada con Box y superposición de texto)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.**

Sí, el ejercicio resuelto fue completado satisfactoriamente. Se implementaron los composables TarjetaLugar (utilizando Row, Column e imagen recortada en círculo) y TarjetaCompacta (utilizando Box para apilar la imagen de fondo, la capa de oscurecimiento scrim y el título flotante). Ambos estilos responden correctamente dentro de la interfaz.



2. **¿Qué combinación de Composables (Row, Column, Box) y Modifiers consideras que fue la mejor opción para cada uno de los dos estilos de tarjeta implementados? Justifica tu respuesta comparando ambos casos.**

Para el Estilo 1 (TarjetaLugar): La combinación de Surface + Row + Column resultó ser la más eficiente. Row organiza de forma natural los elementos continuos en el eje horizontal (imagen circular a la izquierda y bloque informativo a la derecha). La inclusión del Column interno permite agrupar el nombre y la descripción de forma vertical sin distorsionar la alineación ni requerir posiciones absolutas.

Para el Estilo 2 (TarjetaCompacta): La combinación idónea fue Box junto con Modifier.matchParentSize() y Modifier.align(). Debido a que este diseño requiere colocar capas unas sobre otras en el eje de profundidad Z (Imagen -> Capa oscura de contraste -> Texto superpuesto), Box es el único contenedor nativo que permite dicho comportamiento sin recurrir a estructuras de diseño complejas.

3. **¿Qué diferencia encontraste entre construir estas interfaces con Composables y Modifier, en comparación con el enfoque clásico de LinearLayout/ConstraintLayout en XML? Menciona al menos una ventaja concreta que hayas notado.**

La diferencia radica en el paso de un modelo imperativo con archivos declarativos desacoplados (XML) a un paradigma totalmente declarativo basado en código Kotlin modular.

La eliminación de archivos XML auxiliares y la unificación de estado e interfaz. En el enfoque XML tradicional, lograr una tarjeta con imagen de fondo, recortes redondeados y capa transparente requería crear múltiples archivos drawable.xml y coordinar IDs en el Layout. En Jetpack Compose, toda la apariencia, comportamiento e interactividad se definen de manera concisa encadenando modificadores (Modifier.clip(), Modifier.background(), Modifier.clickable()) directamente en la función de UI, facilitando el mantenimiento y la reutilización de código.

CONCLUSIONES

1. Se comprendió la función de los composables estructurales básicos (Row, Column y Box) junto con la cadena de Modifier, evidenciando que Row y Column son ideales para distribuciones secuenciales en dos dimensiones, mientras que Box permite la superposición de elementos en el eje de profundidad (eje Z) para construir layouts complejos.
2. Se logró la implementación práctica de tres estilos de tarjetas (cards) diferenciados para presentar imagen, título y descripción, demostrando la versatilidad de Jetpack Compose para reutilizar componentes y transformar la jerarquía visual de la información según las necesidades de diseño de la aplicación.
3. Se verificó experimentalmente que el orden de encadenamiento dentro del Modifier altera de forma directa el resultado final de la interfaz, puesto que propiedades como .clip(), .background() y .padding() se aplican de manera secuencial, condicionando la geometría, el área de pintado y la zona interactiva del componente.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Android Developers, "Compose layout basics". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/layouts/basics

[2] Android Developers, "Modifiers in Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/modifiers

[3] "POSICIONA elementos con JETPACK COMPOSE de manera FÁCIL", video. [En línea]. Disponible: https://www.youtube.com/watch?v=vTNFb44AHWs

[4] "Usando Cards de Material Design en Jetpack Compose", video. [En línea]. Disponible: https://www.youtube.com/watch?v=iZiXpWRIl3U